



(Assignment #03)

Name: Muhammad Ali

Reg No: FA22-BSE-114

Subject: Software Re-Engineering

Submit To: Mam Hafzah Mehreen

Date: 21-11-2025

Part A: Explain the Following Quality Issues

1. Quality Issues in Reverse Engineering

Reverse engineering involves extracting design, architecture, and business logic from existing source code without adequate documentation. The key quality issues include:

Loss of High-Level Abstraction: The original design intent and architectural patterns are often obscured in the code. Recovering a coherent, high-level model is difficult and error-prone.

Undeclared Dependencies: Legacy systems frequently have hidden dependencies on specific libraries, frameworks, or hardware that are not explicitly stated in the code, leading to incorrect assumptions during recovery.

Obsolete or Dead Code: It's challenging to distinguish between active code and code that is no longer used, which can result in the recovery of an overly complex and inaccurate design model.

Knowledge Gap: The original developers may have left, and their tacit knowledge about "why" certain decisions were made is lost, making the recovered design incomplete or logically flawed.

2. Documentation-Related Issues

Documentation often degrades over a software system's lifecycle, creating significant hurdles during re-engineering.

Outdated Documentation: Documentation does not match the current behavior of the system, leading developers astray and causing them to make incorrect modifications.

Missing Documentation: Critical documents, such as architectural overviews, API specifications, or data dictionaries, are absent, forcing developers to rely solely on code analysis.

Inconsistent Documentation: Different documents (e.g., user manuals, technical specs, code comments) contradict each other, creating confusion about the system's true functionality.

Incorrect Documentation: Documentation contains factual errors, which is worse than having no documentation at all as it actively misguides the re-engineering effort.

3. Violation of Information Hiding

Information hiding is a core principle of good design where internal implementation details of a module are hidden from other modules. Violations include:

Tight Coupling: Modules are highly dependent on each other's internal data structures or logic. A change in one module forces widespread changes in others, making the system rigid and difficult to modify.

Exposed Internal Logic: Critical data is declared as global, or private methods/classes are made public, allowing external components to manipulate internal state directly. This leads to unpredictable side effects and bugs.

Poor Modularization: The code is organized into large, monolithic blocks (e.g., "God classes") with multiple responsibilities, rather than small, focused modules with clear interfaces.

4. Quality Issues in Testing

Inadequate testing practices in the legacy system severely impact the safety and effectiveness of re-engineering.

Missing Test Cases: There are no unit tests for individual components, making it impossible to verify that a small change works correctly in isolation.

Lack of Regression Tests: There is no automated test suite to ensure that new changes do not break existing functionality after refactoring.

Incomplete Coverage: The existing tests only cover a fraction of the codebase, leaving large portions of business logic untested and vulnerable to regressions.

Manual-Only Testing: Reliance on slow, repetitive, and error-prone manual testing processes drastically slows down the re-engineering feedback loop and discourages frequent changes.

5. Approaches to Reduce These Quality Issues

Incremental Re-Engineering: Instead of a complete rewrite, refactor and improve the system in small, manageable steps. This reduces risk and allows for continuous validation.

Establish a Safety Net: Before making significant changes, create a comprehensive suite of automated regression tests (unit, integration, end-to-end) for the critical parts of the system.

Refactoring: Systematically improve the internal structure of the code without changing its external behavior. Focus on breaking down monolithic code, reducing coupling, and enforcing information hiding.

Modern Tooling: Use static code analysis tools to identify code smells, architecture compliance tools to visualize dependencies, and automated refactoring tools in modern IDEs.

Living Documentation: Generate documentation from the code (e.g., using tools like Swagger for APIs) and enforce that it is updated as part of the definition of "done" for any code change.

Continuous Integration (CI): Automate the build, test, and analysis processes to get immediate feedback on the quality impact of every change made during re-engineering.

Part B: Scenario-Based Application

Scenario: Legacy Banking System "BankSys"

1. Scenario Description

BankSys is a core banking application developed over 20 years ago in a legacy language like COBOL. It handles critical operations such as customer accounts, transaction processing, and loan management. The system needs re-engineering for several reasons:

High Maintenance Costs: Finding COBOL developers is expensive and difficult. Even minor changes are risky and time-consuming.

Inability to Integrate: BankSys cannot communicate with modern mobile banking apps, payment gateways, or fraud detection services due to a lack of APIs.

Regulatory Non-Compliance: Implementing new financial regulations requires months of work and extensive manual testing.

Frequent Outages: The monolithic architecture means a small bug in one module can bring down the entire system.

2. Relevant Quality Issues

The most relevant quality issues from Part A for BankSys are:

Violation of Information Hiding: The system is a single, tightly-coupled monolith. For example, the transaction processing logic is directly intertwined with the user interface code and database calls. This makes it impossible to extract and modernize a single service (e.g., a new "Payment Service") without affecting everything else.

Documentation-Related Issues: The original design documents are lost, and the remaining technical manuals are 15 years old, bearing little resemblance to the current system. This forces engineers to reverse engineer everything from millions of lines of COBOL code.

Quality Issues in Testing: There are no automated tests. The only testing is a two-week manual regression process performed before each release, which is completely inadequate for a safe, incremental re-engineering project.

3. Application of Quality Improvement Approaches

To address these issues, the following practical solutions are proposed:

To Address Testing Issues: Establish a Safety Net

Solution: Implement a "humble wrapper" or "strangler fig" pattern. Create a thin layer of modern code (e.g., in Java or Python) that wraps the legacy BankSys modules. Write a comprehensive suite of characterization tests against this wrapper to capture the current observed behavior of the system. This creates a regression test safety net before touching the legacy code.

To Address Information Hiding: Incremental Refactoring and Modularization

Solution: Use the safety net from above to begin incremental refactoring.

Identify a bounded context (e.g., "Calculate Interest").

Refactor the relevant COBOL code to expose a clean interface for this function.

Gradually extract this logic into a separate, independently deployable microservice. This directly attacks the tight coupling and poor modularization by enforcing strong boundaries and information hiding between the new service and the old monolith.

To Address Documentation Issues: Create Living Documentation

Solution: As services are extracted, use API specification tools like OpenAPI (Swagger) to auto-generate interactive documentation for each new service's endpoints. For the core business rules being uncovered during refactoring, use a lightweight wiki (e.g., Confluence) and make updating it a mandatory step in the development workflow for every change related to that service.

4. Impact of the Proposed Solutions

The proposed solutions will significantly improve the BankSys system:

Improved Quality: The automated test safety net catches regressions immediately, preventing new bugs from being introduced. Refactored code with clear interfaces is less prone to errors.

Enhanced Maintainability: The shift from a monolithic to a modular, service-oriented architecture makes the system easier to understand and change. Developers can work on isolated services (like the new "Payment Service") without needing to understand the entire million-line codebase. Living documentation ensures knowledge is preserved and accessible.

Increased Reliability: The decoupled nature of microservices means a failure in one service (e.g., loan calculator) does not necessarily crash the entire banking system. Automated testing and CI/CD pipelines lead to more stable and frequent releases, improving overall system stability and uptime.